UNIVERSITY OF BRISTOL

School of Mathematics

Algorithms and Programming in C(++) and R
MATH10017
(Paper code MATH-10017W)

Winter 2025 1 hours 30 minutes

This paper contains two sections: Section A and Section B.
Each section should be answered in a separate booklet.

Section A contains FIVE questions and Section B contains TWO questions
All SEVEN answers will be used for assessment.

Calculators are not permitted.

Do not turn over until instructed.

Section A: Short Questions

A1. (**8 marks**)

(a) (**2 marks**) In C programming language, which one of the following statements is **wrong**?

(A) Elements in an array are stored in a contiguous section of memory.
(B) The name of an array can be used as a pointer to its first element.
(C) The index of the first element of an array is 1.
(D) Accessing memory outside the bounds of an array may cause undefined behaviour.
(E) C code must be compiled, not interpreted.

Solution

[Bookwork]

[C]

(b) (**3 marks**) In R, what is the main difference between a `vector` and a `list`? Provide a short example to illustrate.

Solution

[Easy but requires some thinking]

[Answer: A vector in R is homogeneous: all elements must have the same type (e.g., numeric, character). Example: `c(1,2,3)`. A list is heterogeneous: elements can be of different types or lengths. Example: `list(1, "a", c(2,3))`.]

(c) (**3 marks**) In R, what is the index of the first element of a vector? Give a short example to illustrate.

Solution

[Very easy check of understanding.]

[Answer: The first element of a vector in R has index 1. For example, `x <- c(10,20,30)` and `x[1]` returns 10.]

A2. (**8 marks**) Read the following C program and answer questions.

```
#include <stdio.h>

void main(){
    int f(int a, int x){
        printf("function f\n"); // L1
        return a + 2*x;
    }
    int a = 2, b = 1;
    int ff = f(a, f(b, a)); // *
    printf("%d\n", ff); // *

}
```

(a) (**3 marks**) What will happen if you try to compile this code? Explain the issue, and propose a fix. You are not allowed to change the lines marked by *. Your fix should also not alter the intended outcome of the code.

Solution

[Very similar question seen in lectures or homeworks.]

[Answer: The code will not compile. Move the f function outside of the main function.]

(b) (**2 marks**) After the fix, what is the output of the program?

Solution

[Very similar question seen in lectures or homeworks.]

[Answer:]

function f

function f

12

(c) (**3 marks**) Draw the stack memory layout when the program running the like marked with the comment "L1" **the second time**. Your diagram must include all local variables and their values.

Solution

[Unseen, similar to a homework question]

[Answer:]

```
Two boxes.
Top, f. a = 2, x = 5
Bottom, main. a = 2, b = 1.
```

A3. (**8 marks**)

The following C code sums overs all rows of a 3 by 3 matrix. The matrix is stored in `A` in row major order. It prints out a column vector, for which the i-th element is the sum of the i-th row.

```
int elem(int M[], int i, int j){
    __YOUR CODE HERE__
}
void rowsum(int A[]){
    int sum[3] = {0, 0, 0};
    for (int i = 0; i < 3; i++){
        for (__YOUR CODE HERE__){
            sum[i] += __YOUR CODE HERE__;
        }
    }

    //print the result
    for (int i = 0; i < 3; i++){
        printf("%d \n", sum[i]);
    }
}

void main(){
    // A stores a 3 by 3 matrix in row-major order.
    int A[] = {1,2,3, 2,1,3, 3,3,1};

    rowsum(A);
}
```

(a) (**2 marks**) Suppose that the `elem` function returns the i, j-th element of a 3 by 3 matrix `M` stored in row-major order. Complete `elem` by using **one line of code**. Use zero-based indexing.

Solution

[bookwork]

[Answer:]

```
return M[i*3 + j];
```

(b) (**3 marks**) Complete the missing lines in the `rowsum` function.

Solution

[Very similar question seen in lectures or homeworks.]

[Answer:]

```
for (int j = 0; j < 3; j++){
  sum[i] += elem(A, i, j)
```

(c) (**3 marks**) Suppose I use the above algorithm to compute the row sum of an m by n matrix. It takes t seconds to compute the outcome. Approximately, how much time would the algorithm take when computing row sum on a $2m$ by $2n$ matrix? Briefly justify your answer.

Solution

[Unseen, challenging]

[Answer:]

```
4t,
the computational complexity of is O(num_row*num_col)
```

A4. (**8 marks**) Consider an R function called `nearest_index` which takes as inputs a scalar number `x` and a numeric vector `v`. The function should return the index `k` of the element in `v` that is **closest** to `x`. For example:

```
> z <- c(3, -10, 7)
> nearest_index(x = -12, v = z) # closest to -12 is -10 (index 2)
[1] 2
> nearest_index(x = -1, v = z)  # closest to -1 is 3 (index 1)
[1] 1
```

Read the following R code and answer the questions.

```
nearest_index <- function(x, v){
  ng <- length(v)
  best_k <- 1
  best_d <- Inf # + Infinity

  for(ii in 1:ng){
    d <- abs(x - v[ii])
    if (d < best_d) {
    # YOUR CODE HERE
    }
  }

  return(best_k)
}
```

(a) **(3 marks)** Complete the function so that it works as described above.

Solution

[Routine.]

```
nearest_index <- function(x, v){
  ng <- length(v)
  best_k <- 1
  best_d <- Inf

  for(ii in 1:ng){
    d <- abs(x - v[ii])
    if (d < best_d) {
      best_d <- d    # <- We just added these 2 lines
      best_k <- ii   # <-
    }
  }
  return(best_k)
}
```

(b) **(3 mark)** Explain what will be the output of this code:

```
x <- c(2, 1)
for(ii in x){
 print( nearest_index(x = ii, v = c(-1, 3, 0)) )
}
```

Solution

[Very simple check of understanding.]

[The code will print out the numbers 2 and 3.]

(c) **(2 marks)** Assume that we know that the elements of `v` are increasing (i.e., `v[1] < v[2] < ...`). Make your function more efficient by exploiting this fact (it should be sufficient to add one line of code).

Solution

[Unseen, requires some thinking.]

```
nearest_index_ord <- function(x, v){
  ng <- length(v)
  best_k <- 1
  best_d <- Inf

  for(ii in 1:ng){
    d <- abs(x - v[ii])
    if (d < best_d) {
      best_d <- d
      best_k <- ii
    }
    if(v[ii] > x) break # <- we have simply added this line
  }
  return(best_k)
}
```

A5. (**8 marks**) The Frobenius norm of a matrix $A \in \mathbb{R}^{m \times n}$ is

$$\|A\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} A_{ij}^2}.$$

Answer the following questions in R.

(a) (**4 marks**) Write an R function `frobenius_norm(A)` that computes the Frobenius norm of a numeric matrix A using at least one `for()` loop and without calling `norm()`.

Solution

[Unseen, but student have seen very similar computations.]

```
frobenius_norm <- function(A){
  m <- nrow(A); n <- ncol(A)
  s <- 0
  for (i in 1:m){
    for (j in 1:n){
      s <- s + A[i, j]^2
    }
  }
  return( sqrt(s) )
}
```

(b) (**2 marks**) Write a vectorised version of `frobenius_norm(A)` that avoids explicit loops or calls to `sapply`, `lapply`, etc.

Solution

[Unseen, but students are expected be familiar basic R's vectorised operations.]

```
frobenius_norm_vec <- function(A){
  sqrt( sum(A^2) )
}
```

[But other solutions, e.g. using rowSums, are acceptable.]

(c) (**2 marks**) Explain what, in your opinion, will be the output of this code:

```
X <- matrix(c(1, 2, 2, 1), ncol = 2, nrow = 2, byrow = TRUE)

z <- frobenius_norm(X - t(X))

z:10
```

Solution

[Unseen, challenging]

[A vector with elements 0, 1, ..., 10.]

Section B: Longer Questions

Please use a new booklet for this Part.

B1. (**30 marks**) Consider a meteorological dataset that contains the record of rainfall of m cities on n dates. The dataset is stored as a matrix $A \in \{0,1\}^{m \times n}$. $A_{i,j} = 1$ indicates that it rained in the city i on the j-th day. $A_{i,j} = 0$ means the city i did not rain on the j-th day.

- Suppose $m = 100$, $n = 365$. Use zero-based indexing in this question.
- You are given the skeleton C code below.
- Function `elem` has been provided to you. `elem(M, i, j)` returns $M_{i,j}$.

```
int elem(int M[], int i, int j){
  // function body omitted
}

// YOUR CODE GOES HERE

void main(){
    // A is a 100 by 365 matrix stored in row-major order.
    int A[] = {1,0,0,1,0,1,1,0, /*the rest of A is omitted*/};

    int i = 23, j = 17;
    // predict the rain fall of city i on the j-th date
    int rain = predict(A, i, j);
    printf("%d\n", rain);
    // ... other function calls
}
```

(a) (**10 marks**) Write a function `int raininess(int A[], j)`. It returns the total number of cities that rained on the j-th day.

Solution

[seen and similar to homework.]

[Answer:]

```
int raininess(int A[], int j){
    int count = 0;
    for (int i = 0; i < 100; i++){
        if (elem(A, i, j) == 1){
            count++;
        }
    }
    return count;

}
```

(b) (**10 marks**) Write a function `int predict(int A[], i, j)` that returns the prediction of rainfall of city i on the j-th day ($j \geq 7$).

The prediction rule is: city i is predicted to have rain (return `1`) if it rained on at least 3 of the previous 7 days ($j-1, j-2, \ldots, j-7$). Otherwise return `0`.

Solution

[unseen, moderate difficulty.]
[Answer:]

```
int predict(int A[], int i, int j){
    int rainy_days = 0;
    for (int d = j-7; d < j; d++){
        if (elem(A, i, d) == 1){
            rainy_days++;
        }
    }
    if (rainy_days >= 3){
        return 1;
    } else {
        return 0;
    }
}
```

(c) (**10 marks**) Write a function `int sunniest_day(int A[])` that returns the sunniest day of the year. The sunniest day is defined as the day on which the smallest number of cities experienced rain. If there are multiple "sunniest days", printing any one of them is OK. NOTE: You can use functions in previous questions even if you did not answer them.

Solution [unseen, moderate difficulty.]
[Answer:]

```
int sunniest_day(int A[]){
    int mincities = 101; // max number of cities is 100
    int sunniest = -1;
    for (int j = 0; j < 365; j++){
        int numcities = raininess(A, j);
        if (numcities < mincities){
            mincities = numcities;
            sunniest = j;
        }
    }
    return sunniest;
}
```

B2. (**30 marks**) A number d is said to be a *divisor* of a if d divides a exactly. A *perfect number* is a positive integer equal to the sum of its *proper divisors* (its positive divisors excluding itself). For example, 6 is perfect because its proper divisors are 1, 2, and 3, and $1 + 2 + 3 = 6$. A number a is *abundant* if the sum of its proper divisors is greater than a, and *deficient* if it is smaller.

In this question will write R functions to work with these definitions. You may find useful that the R's modulo operator works as follows:

```
> 2 %% 2
[1] 0
> 6 %% 3
[1] 0
> 3 %% 2
[1] 1
> 3 %% 4
[1] 3
> 9 %% 2:9
[1] 1 0 1 4 3 2 1 0
```

(a) (**8 marks**) Write an R function called `is_pd(a, d)` that takes two positive integers `a` and `d` and returns `TRUE` if `d` is a proper divisor of `a`, and `FALSE` otherwise.

Solution

[Bookwork, similar code seen in a lab session. No points are lost for not checking that d is positive (we can assume this).]

```
is_pd <- function(a, d){
  return( (a %% d == 0) & (d < a) & (d > 0) )
}
```

(b) (**8 marks**) Using your function from question (a), write a function `sum_pds(a)` that returns the sum of all proper divisors of the positive integer `a`. Your implementation should include either a `for` or a `while` loop.

Solution

[Unseen, moderately difficult]

```
sum_pds <- function(a){
  total <- 0
  for(d in 1:(a-1)){
    if(is_pd(a, d)){
      total <- total + d
    }
  }
  return(total)
}
```

(c) (**5 marks**) Write a function `classify_number(a)` that returns the string `"perfect"`, `"abundant"`, or `"deficient"` according to the value of `a` and the result of `sum_pds(a)`. Briefly explain how your function handles the case `a = 1`.

Solution

[Unseen, but quite easy]

```
classify_number <- function(a){
  s <- sum_pds(a)
  if(s == a){
    return("perfect")
  } else if(s > a){
    return("abundant")
  } else {
    return("deficient")
  }
}
```

[Edge case: For $a = 1$, the proper divisor set is empty, so the sum is 0. Since $0 < 1$, the function returns "deficient".]

(d) (**5 marks**) Define a function `classify_numbers_v(v)` that takes a vector of positive integers `v` and returns a character vector with the corresponding classification for each element of `v`. Your solution for this part should avoid explicit `for`/`while` loops, but you may use `sapply`, `lapply`, etc.

Solution

[Unseen, but otherwise bookwork]

```
classify_numbers_v <- function(v){
  sapply(v, classify_number)
}
```

(e) (**4 marks**) Write a vectorised version of the function in question (b), called `sum_pds_v(a)`, that computes the sum of proper divisors of a scalar integer `a` without using any explicit `for`/`while` loops or calls to `sapply`, `lapply`, etc.

Solution

[Unseen, fairly challenging]

```
sum_pds_v <- function(a){
  d_v <- 1:(a-1)
  total <- sum( d_v[is_pd(a, d_v)] )
  return(total)
}
```